Experiment No. 6: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE

Aim

To create, compile, deploy, and perform transactions on a **Ballot smart contract** using Solidity and Remix IDE by changing the proposal name from `bytes32` to `string`.

Theory

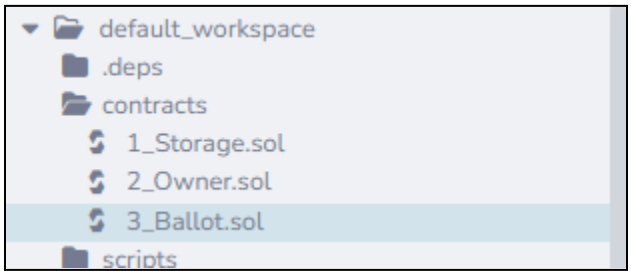
A **smart contract** is a program stored on a blockchain that automatically executes according to predefined rules. Solidity is used to develop Ethereum smart contracts, while Remix IDE provides an environment to write, compile, and deploy them.

The **Ballot contract** implements a voting system in which voters can be given voting rights and can vote for different proposals.

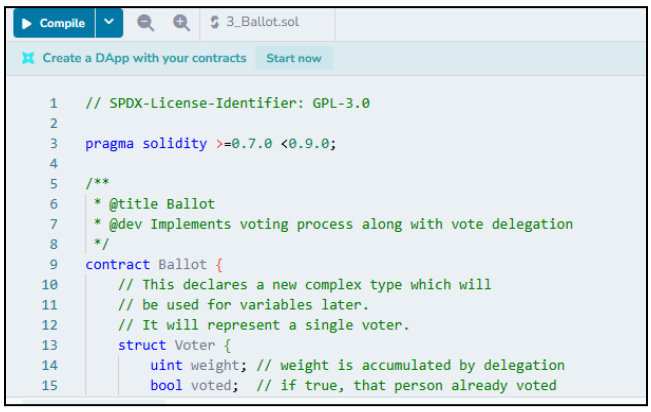
- **`require`** checks conditions and reverts the transaction if the condition is not satisfied.
- **`mapping`** stores voter information associated with Ethereum addresses.
- **`storage`** stores data permanently on the blockchain.
- **`memory`** is temporary storage used during function execution.
- **`bytes32`** is fixed-size and efficient, while **`string`** allows readable proposal names of variable length.

Steps / Procedure

1. Open **Remix IDE**.
2. Open the **contracts** folder.



3. Open the **`3_Ballot.sol`** contract.



4. Understand the structure and functions of the Ballot contract.
5. Change the proposal name from `bytes32` to `string` and Change the `Proposal` structure to use:
   string name;

```
    // This is a type for a single proposal.
    struct Proposal {  //Khushi Singh, rno 30
    string name;
    uint voteCount;
}
```

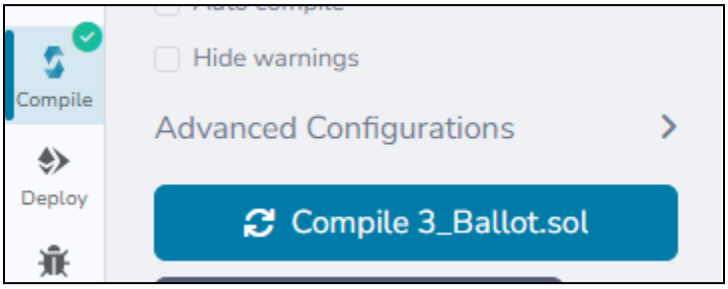
6. Change the constructor parameter from:
   constructor(bytes32[] memory proposalNames)
   to:
   constructor(string[] memory proposalNames)

```
constructor(string[] memory proposalNames) {
```

7. Modify `winnerName()` to return `string memory`.

```
function winnerName() external view returns (string memory winnerName_)
{
    winnerName_ = proposals[winningProposal()].name;
}
```

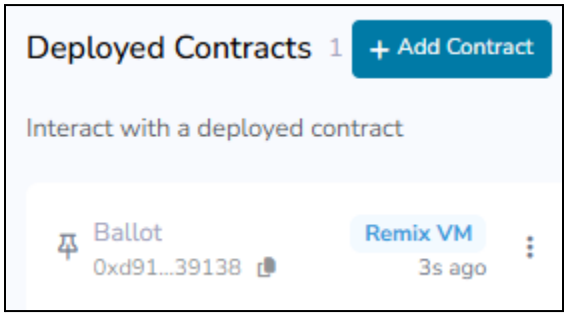
8. Compile the contract using the **Solidity Compiler**.



9. Open **Deploy & Run Transactions**.
10. Select **Remix VM** as the environment.
11. Select the **Ballot** contract.
12. Enter proposal names



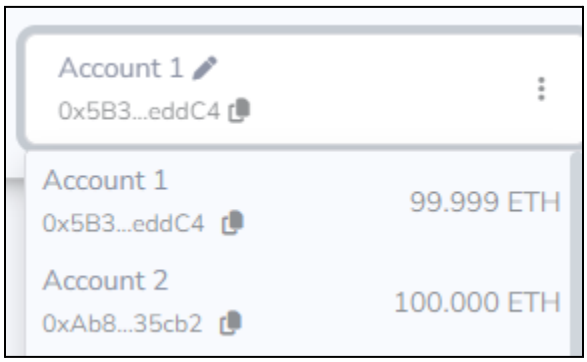
14. Click **Deploy** and verify that the contract appears under **Deployed Contracts**.



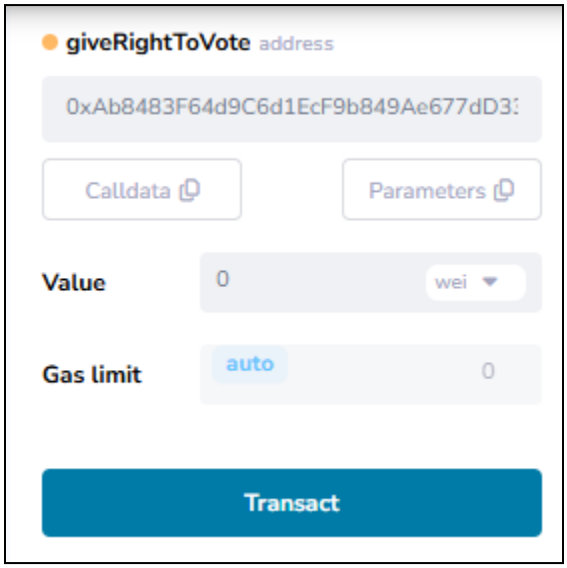
15. Use the `proposals` function and enter indices `0`, `1`, and `2` to view the proposal details.



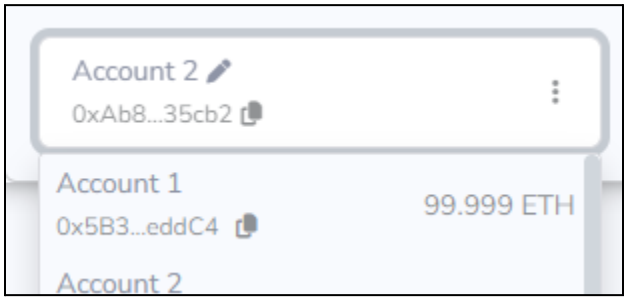
16. Copy the **Account 2 address** from the Account list.



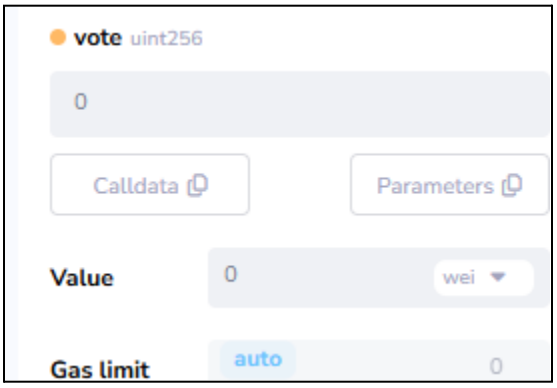
17. Switch back to **Account 1 (chairperson)**.
18. In the deployed Ballot contract, enter the copied Account 2 address in **`giveRightToVote`** and click **transact**.



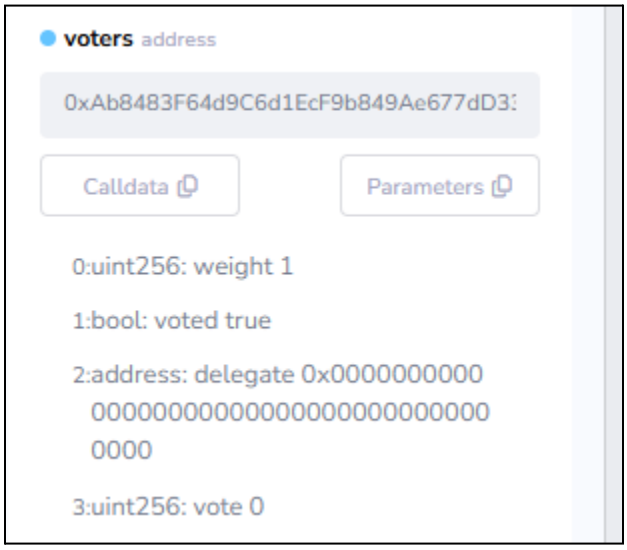
19. Switch the selected account to **Account 2**.



20. In the **`vote`** function, enter the proposal index:



21. Click **transact** to cast the vote for the first proposal.
22. Use the **`voters`** function with Account 2's address to verify that the vote was recorded.



The contract uses the chairperson to grant voting rights, and the voter information is maintained through the voter mapping.

Output

- `3_Ballot.sol` was successfully compiled.
- The Ballot smart contract was successfully deployed.
- Proposal names **Khushi Singh, Bob, and Charlie** were loaded.
- Proposal details were viewed using their indices.
- Account 2 was given the right to vote by the chairperson.
- Account 2 successfully voted for proposal **0 (Khushi Singh)**.
- Voter information was checked to verify the recorded vote.

Conclusion

The Ballot smart contract was successfully **compiled, deployed, and tested** using Solidity and Remix IDE. The experiment demonstrated proposal creation, granting voting rights to another account, and casting a vote. It also provided practical understanding of `require`, `mapping`, `storage`, `memory`, and the difference between `bytes32` and `string` in Solidity.